Variables de entorno y primera petición HTTP autenticada (Bloque 4) / Variables de entorno: cómo excluir tus secretos del repo

✓ **Hecho:** Ver

UNIDAD U1.2 · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Variables de entorno: cómo excluir tus secretos del repo

Tu repositorio ya tiene estructura base, **entorno virtual** y dependencias documentadas, verificadas además por un compañero o compañera en el bloque anterior.

Falta el último elemento: que tu proyecto pueda usar credenciales reales (la llave de API de tu proveedor de modelo) sin que esa credencial quede jamás escrita en el código ni subida al repositorio.

En el **Bloque** 2 viste por qué esta regla no tiene excepciones; **esta lección va al cómo**, en tu propio repositorio.

🎯 Qué proveedor de modelo usar, y cómo conseguir tu llave

Antes de crear tu archivo "**.env**" necesitas algo que guardar ahí: una credencial real de un proveedor de modelo de lenguaje. No todos los proveedores ofrecen lo mismo para empezar, así que vale la pena elegir con criterio en vez de registrarte en el primero que aparezca.



Google Gemini API

Opción recomendada para este programa

Tiene un nivel gratuito permanente (no es una prueba por tiempo limitado) y no pide tarjeta de crédito para activarlo. Es la opción recomendada por defecto para este programa: genera tu llave en **Google AI Studio** y con eso ya tienes lo que necesitas para el resto de esta unidad.



Groq

Alternativa gratuita de alta velocidad

También tiene un nivel gratuito permanente, sin tarjeta, con límites de solicitudes por minuto y por día según el modelo. Es una alternativa válida si prefieres explorar modelos open-source servidos a alta velocidad.



Anthropic (Claude)

Créditos gratuitos limitados

Da una cantidad limitada de créditos gratis, de una sola vez, al registrar una cuenta nueva: no es un nivel gratuito permanente. Una vez que se agotan, pasa a ser de pago.



Mistral AI

Créditos gratuitos ilimitados

A diferencia de Anthropic, sí ofrece un nivel gratuito permanente: su plan "Free" incluye créditos de API que se renuevan cada mes (no es un crédito único de bienvenida), aunque con límites de uso más ajustados que los planes pagos.



OpenAI

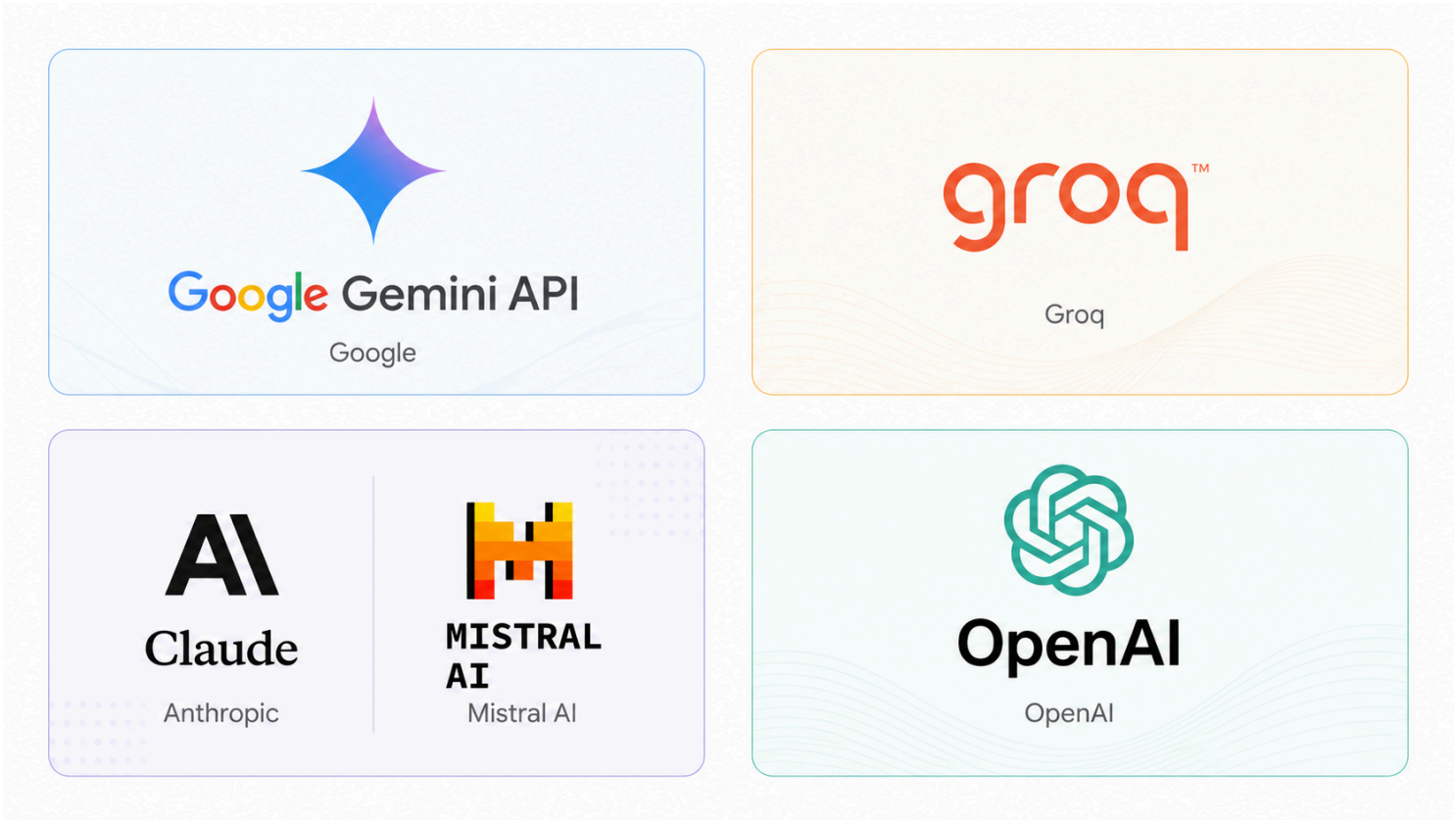
API de pago desde el primer uso

No ofrece nivel gratuito ni créditos de bienvenida para su API de desarrolladores: es un servicio distinto de **ChatGPT**, y se cobra desde el primer token que uses.

Hugging Face

Acceso a miles de modelos abiertos

Crear una cuenta y generar tu token de acceso es gratis y no pide tarjeta, pero el crédito de inferencia mensual que incluye la cuenta gratuita es muy pequeño (una fracción de dólar), pensado más para probar que para sostener el uso de toda la unidad. Vale la pena conocerlo porque da acceso a miles de modelos abiertos distintos, pero no lo elijas como tu proveedor principal si buscas algo sin fricción.



⭐ **Si no tienes una preferencia previa, usa Google Gemini API**: es la ruta con menos fricción para este programa, porque no depende de que se agoten créditos ni de ingresar un medio de pago.

Genera tu llave desde el sitio del proveedor que elijas, y guárdala aparte por un momento (no la pegues todavía en ningún archivo de tu repositorio): en la siguiente sección la vas a guardar donde corresponde.

🔄 De la teoría a tu propio archivo .env

Tu plantilla de repositorio base ya trae un archivo llamado "**.env.example**": una copia de referencia, sin ningún valor real, que muestra qué variables necesita tu proyecto. Tu tarea es crear, a partir de esa referencia, tu propio archivo "**.env**" (sin el ".example" al final) y completar ahí tus credenciales reales.

```
# macOS / Linux / WSL
cp .env.example .env

# Windows (PowerShell)
Copy-Item .env.example .env
```

La diferencia entre ambos archivos es exactamente la diferencia entre lo que se sube al repositorio y lo que no:



.env.example

".env.example" sí se sube, porque no contiene ningún secreto, y sirve para que cualquier persona sepa qué variables necesita configurar.



.env real

Tu **".env"** real nunca se sube, y por eso ya viene excluido en el **".gitignore"** de tu repositorio desde el **Bloque 2**.

Si ya no tienes a mano la plantilla que descargaste en el Bloque 2, o necesitas volver a descargarla para revisar tu ".env.example", también puedes obtenerla desde este enlace directo ➡️ **Plantilla Repositorio Base**

Es normal, y esperado, ver "**.env.example**" aparecer en tu "**git status**" y después en tu repositorio de **GitHub**: eso no es una filtración de secretos, es justamente la plantilla de referencia cumpliendo su función. No lo confundas con tu ".env" real: si ves el archivo ".env" (sin ".example") en esa misma lista, ahí sí hay que detenerse antes de hacer commit.



Concretamente, esto es lo que deberías ver en cada archivo. Tu "**.env.example**" (el que sí se sube al repositorio) trae solo el nombre de la variable, sin ningún valor real:

```
GEMINI_API_KEY=tu_llave_aqui_como_ejemplo
```

Y tu "**.env**" real (el que nunca se sube) trae ese mismo nombre de variable, pero con tu credencial real como valor:

```
GEMINI_API_KEY=AIzaSyD_tu_llave_real_completa_aqui
```

El nombre de la variable, a la izquierda del "=", es idéntico en los dos archivos; lo único que cambia es el valor a la derecha. Si tu proveedor no es Google Gemini, el nombre de tu variable va a ser distinto: usa el que ya viene definido en tu propio ".env.example", no el de este ejemplo.

El nombre de cada variable importa tanto como su valor. "**.env.example**" ya trae el nombre exacto que tu código va a buscar (algo como la variable de la llave de API de tu proveedor de modelo); si en tu "**.env**" real le pones un nombre distinto, aunque sea por una letra, tu código no la va a encontrar, y el error que vas a ver no va a decir "**nombre de variable equivocado**": va a decir algo más genérico, como que la credencial no existe o vino vacía. Cuando eso pase, lo primero que hay que comparar es que el nombre de la variable en tu ".env" coincida exactamente con el que aparece en ".env.example".

💻 Cómo tu código llega a usar esas variables

☝️ Un archivo "**.env**" por sí solo no le sirve de nada a tu programa si nada lo lee.

La forma estándar de conectar ambas cosas es una librería que carga ese archivo al iniciar tu programa, y deja sus valores disponibles como variables de entorno del sistema operativo, exactamente como si las hubieras definido a mano en la terminal.

La librería estándar para esto en Python es "python-dotenv". Se instala como cualquier otra dependencia de tu proyecto, con tu entorno virtual activo:

```
pip install python-dotenv
```

Si quieres confirmar ahora mismo, por tu cuenta, que tu ".env" se está cargando bien, puedes ejecutar esto en una consola de Python dentro de tu entorno virtual:

```
from dotenv import load_dotenv
import os

load_dotenv()
print(os.getenv("GEMINI_API_KEY") is not None)
# o el nombre de tu variable, si tu proveedor no es Gemini
```

Esto solo confirma que la variable se cargó ("True" o "False"): no es el código que vas a usar para hacer la petición a la API, eso lo escribes recién en la Unidad 1.3.

Desde ese momento, tu código pide la credencial por su nombre de variable, no por su valor: en ningún lugar del código aparece la llave real escrita, solo el nombre de la variable donde se espera encontrarla en tiempo de ejecución.

Esa indirección es justamente lo que hace posible que el mismo código funcione en tu computador, en el de un compañero, o en un servidor de producción más adelante en el programa, cada uno con su propio archivo ".env" y sus propias credenciales, sin tocar una sola línea de código.



Esta separación entre código y credenciales es la misma idea que vas a encontrar, a mayor escala, cuando en el Módulo 3 tu aplicación deje de correr solo en tu computador y empiece a correr también en un contenedor, y eventualmente en un entorno de despliegue distinto.

Cada uno de esos entornos (tu computador, el contenedor, un eventual servidor) va a tener su propio archivo de variables, con sus propias credenciales, mientras el código que las usa se mantiene exactamente igual.

> 🧭 Aprender a pensar así desde ahora, con un solo archivo "**.env**" y una sola credencial, es lo que hace que esa transición más adelante no requiera rediseñar nada.

Vale la pena entender también qué pasa si te saltas este paso. Si tu código intenta leer una variable de entorno que nunca se cargó (porque olvidaste crear el ".env", porque el nombre no coincide, o porque la librería que lo carga nunca se ejecutó), lo más probable es que la credencial llegue vacía o inexistente, y que el error aparezca recién en el momento de conectarte a la **API**, no antes.

Por eso conviene verificar que la variable efectivamente tiene un valor antes de intentar cualquier

petición: ahorra tiempo de diagnóstico frente a un error de conexión que en realidad es un problema de configuración.

🛡️ Por qué esta regla se trata con tanta seriedad

No es una precaución exagerada. Existen sistemas automatizados que rastrean constantemente los repositorios públicos de GitHub buscando patrones que parecen credenciales expuestas, y cuando encuentran una, pueden empezar a usarla en minutos, no en días.



Una llave de **API** de un proveedor de modelo, en manos de alguien más, se traduce directamente en cargos a tu nombre o al de la organización a la que pertenezca esa credencial, además de dejar sin efecto cualquier control de acceso que se haya definido para ella.

Esta es la razón de fondo por la que la exclusión de secretos aparece explícitamente en la lista de cotejo del **checkpoint** de esta **unidad**: **no es un detalle de estilo, es una condición de seguridad real.**

🔎 Verificar que el secreto realmente quedó fuera del repositorio

No basta con crear el archivo ".env" y asumir que la exclusión funcionó: verifícalo activamente antes de seguir.



Revisa el estado de tu repositorio

Revisa el estado de tu repositorio y confirma que **".env"** no aparece listado como un archivo nuevo o modificado. Si aparece, la exclusión no está funcionando y hay que corregirla antes de continuar.

Si la exclusión está funcionando, "git status" no debería mostrar tu ".env" en ningún lado:

```
git status
On branch main
nothing to commit, working tree clean
```

Si en cambio ves algo como esto, la exclusión no está funcionando:

```
git status
On branch main
Untracked files:
(use "git add <file>..." to include in what will be committed)
    .env
```



Abre tu ".gitignore"

Abre tu ".gitignore" y confirma que la línea que excluye **".env"** sigue ahí, sin errores de escritura (un espacio de más, una letra distinta, pueden hacer que la exclusión no aplique).



Si ya subiste un ".env"

Si en algún momento anterior llegaste a subir un ".env" por error, avísale al equipo docente de inmediato: agregarlo ahora al ".gitignore" no borra lo que ya quedó en el historial, y ese caso necesita un tratamiento aparte para limpiar el historial completo.

⚠️ Si tu código no encuentra la variable



`os.getenv()` **devuelve** `None`

Casi siempre significa que `load_dotenv()` nunca se ejecutó antes de leer la variable, o que se ejecutó desde una carpeta distinta a la de tu `.env`. Confirma que la llamada a `load_dotenv()` ocurre al principio de tu script.



El archivo queao guaraaao como `.env.txt`, no como `.env`

Algunos editores agregan una extensión automáticamente al guardar un archivo con un nombre que no reconocen. Confirma el nombre exacto en tu explorador de archivos o con un listado de la terminal, no solo por lo que muestra el editor.



El nombre de la variable no coincide exactamente

Una letra de más, una mayúscula distinta, o un espacio alrededor del `=` hacen que no se reconozca. Compara el nombre en tu `.env` contra el de tu `.env.example` carácter por carácter.



Error `No module named 'dotenv'` al ejecutar tu script

Significa que `python-dotenv` no está instalado en el entorno virtual que está activo en ese momento. Confirma con `pip install python-dotenv`, con tu entorno virtual activo.



Siguiente lección

Bloque 4 · Tu primer disparo a una API

Con tu archivo **".env"** creado, completado con tu credencial real, y confirmado como excluido del repositorio, ya tienes todo lo necesario para el último tramo de esta unidad: usar esa credencial para hacer una petición real a una **API**, en la siguiente lección de este mismo bloque.

Última modificación: jueves, 24 de septiembre de 2026, 11:50